Mise, from camera to receipt

Start here before rehearsing the live demo.

At closing time, a cook knows what the kitchen needs. The buyer still has to read the handwriting, recognize the ingredient in a supplier's catalog, and decide how many whole packs to buy. Mise carries the cook's original wording through those decisions.

Photo > Verified text > Ingredient > Supplier pack > Receipt

Follow one order

The sample note asks for one case of eggs and one case of oat milk in Chinese. The buyer can review in English. After checking the transcription, they see two demo products. The catalog lists 180 eggs for $54 and six cartons of oat milk for $24. Approving saves a $78 simulated receipt.

Three places the work happens

Browser

Shows the original image, collects corrections, displays the cart and asks the merchant to approve.

Next.js server

Reads the photo, reconciles the two readings, interprets the note, looks up eligible product candidates, calculates packs and prices, and processes approval.

SQLite

Saves the versioned draft and its final receipt on the demo host. The approval ends here. No payment or supplier submission takes place.

How to use this guide

Pages 2–6 follow the software in execution order. Page 7 is a live demo script. Page 8 has short answers for likely judge questions. The $78 example is reproducible from the bundled printed note. Use an actual handwritten list to demonstrate handwriting performance.

The stack and the browser

One full-stack application serves the phone interface and its JSON API.

Next.js 16 + React 19

The App Router serves the merchant page at /. The root layout sets document metadata and loads styles. The interactive page uses React state to switch between scan, cart, and receipt. These are three views of the same URL.

TypeScript + Node.js

Strict TypeScript describes the objects that cross the browser/server boundary. Node.js 24+ runs the routes, OCR child process, network requests and built-in SQLite driver. npm manages dependencies. The dev launcher locates a compatible Node installation, including on Windows.

CSS + browser APIs

Responsive CSS adapts the layout for phones, light/dark preferences, and reduced motion. Fontsource bundles Montserrat, Merriweather and Ubuntu Mono. Native file inputs handle camera capture and uploads. Canvas handles image resizing and cropping. Fetch sends JSON to routes on the same origin.

Language support

The interface has hand-authored English, Spanish and Simplified Chinese copy. The note language is a separate setting and also supports Traditional Chinese. Ingredient normalization, translated interface labels and photo transcription are three different pieces of code.

What the browser does with a photo

The buyer enters a US ZIP or ZIP+4 and selects a JPEG, PNG or WebP image up to 20 MB. The browser decodes it, reduces the longest side to at most 2,200 pixels and compresses to JPEG within the four-million-character encoded limit. A crop can be selected by dragging or adjusting four edges. The full image remains available for comparison.

The browser keeps the photo, editable text and current view in React state. A reload loses the current view. A fresh list clears the photo and cart and retains the review language and ZIP. The page uses status announcements, labeled fields and focus management during transitions.

How Mise reads handwriting

Two readings expose disagreements the merchant can resolve.

1. Read the selected image locally

POST /api/transcribe receives the image and source-language hint. Node starts a short-lived Tesseract.js 7 worker on the server. It uses bundled English, Spanish, Simplified Chinese or Traditional Chinese language data. The worker returns text, the pixel coordinates of each line and a confidence score that includes the weakest word. The server caps local work at one scan at a time per process and 18 seconds per attempt.

2. Verify the image online

The server sends the image and, when available, Tesseract's draft to a vision model through OpenRouter. OpenRouter routes the request. The local configuration selects Google Gemini 2.5 Flash. The prompt asks the model to copy what is visible, preserve mixed scripts, quantities and units, mark unreadable text with [?], and treat writing on the image as data. The model and API key are configured server-side.

3. Align the lines

The code normalizes superficial differences such as spaces between Chinese characters. A longest-common-subsequence algorithm anchors lines that agree and pairs the gaps between them. If one reader misses the middle line of a list, later matching lines can still align. A difference, missing reading, unclear character or local confidence below 75 (or unavailable) triggers review. The threshold determines when to ask for review.

4. Let the buyer resolve uncertainty

The buyer can inspect the image strip when coordinates exist, select a reading, reread the crop, type a correction, or omit the line. Every flagged row needs a decision before the reviewed text is accepted. The whole transcription is then editable. If one engine fails, its counterpart can supply text for review. If neither yields a transcription, the buyer can type from the photo or retry.

> ### Why two readers?
>
> Changing "1 case" to "7 cases" matters even when the ingredient name is perfect. Retaining the image and surfacing disagreement gives the buyer a concrete way to catch that error.

From words to ingredients

After transcription, purchasing decisions are made by TypeScript rules.

Separate the line, the ingredient and the product

Add to cart sends the verified text, review language and ZIP to POST /api/drafts. The server splits the note into lines, preserves decimal commas and separates recognized recipe headings. Each ingredient keeps the submitted source text, including its original language. A draft supports up to 5,000 characters and 50 source entries.

The matcher normalizes case, accents and spacing and consults a curated vocabulary of 32 ingredient entries. It handles aliases, prefix or suffix quantities, fractions, mixed numbers, a limited set of number words and Chinese numerals, and familiar units or classifiers. A dozen becomes 12 individual items. An exact known alias can identify an ingredient. Partial and one-edit spelling matches produce suggestions that need clarification.

> ### Read this example aloud
>
> “1/2 cebolla picada” is half a chopped onion. The parser identifies onion, keeps the preparation instruction and represents the requested amount as 0.5 of an onion. If the store offers single onions, one whole onion covers the request with half an onion left over.

Why the catalog is smaller than the vocabulary

An ingredient name tells the cook what food they need. A stock keeping unit (SKU) identifies a specific sellable product. The local demo catalog has four SKUs. Case prices are $54 for 180 eggs, $24 for six oat milk cartons, $32 for 25 lb of tomatoes and $28 for 12 units of romaine. These figures are illustrative.

If a catalog match covers an exact whole number of cases, it is ready immediately. Other lines initially need review. During cart creation the server tries research for those lines, then falls back to a usable local pack, rounding up where the units allow it. Otherwise the line is marked skipped and excluded from the total. The cart still shows the source line.

Recognition has deliberate limits. Missing units, ranges, unknown names, similar spellings and unreadable characters can prevent a confident interpretation. Recipe headings are pattern-based, so an arbitrary title in any language is not guaranteed to be detected.

How supplier research works

Compare the actual purchase required to cover the requested amount.

Discovery

For a recognized ingredient, the server searches by its English name at four configured US storefronts, including Ginger Fresh, Palouse Brand, Country Life Natural Foods and Elmhurst 1925. It reads public Shopify-style search suggestions, cart currency and selected product-variant details, including observed prices and pack sizes. It timestamps the evidence. The server processes up to two ingredient lines at once and queries the four stores in parallel, with timeouts and partial results.

Title checks reject incompatible forms, such as onion powder for onion or chocolate oat milk for plain oat milk. If no compatible storefront listing survives the filters, the server tries Open Food Facts. That source can identify a product or pack, but supplies no purchase price or stock signal here. Its listings cannot directly price a cart line.

Comparison

The ranking code converts compatible units within a dimension, such as pounds to kilograms. It avoids guessing how many grams are in an unspecified carton. When available, pack size, whole-pack rounding, purchase increments, order minimums and maximums determine the number of packs. The algorithm considers ingredient fit, information quality, comparable cost and surplus. Up to five candidates are returned.

> ### A worked example from the tests
>
> The kitchen asks for 2 kg of green lentils. One 4 lb pack is about 1.81 kg, so the test buys two packs at $18.95 each for $37.90. A single 18 lb pack covers the request too, but costs $90. These prices are test inputs.

What the ZIP actually does

The ZIP travels with the draft and the research request. The storefront adapter does not request a carrier quote or verify shipping eligibility at that ZIP. It leaves delivery, fees and minimum spend unknown when evidence is missing. The ranker distinguishes provisional known cost from verified delivered cost. The automatic cart can accept a comparable provisional offer and totals pack prices. Calling that a guaranteed delivered price would overstate the implementation.

The API and the database

The server owns quantities, totals, draft versions and the final receipt.

POST /api/transcribe

Accepts the image and source language. Returns transcription, per-line evidence and reader availability.

POST /api/drafts

Accepts verified text, review language and ZIP. Returns the persisted, versioned draft with recommended lines, total and approval status.

POST /api/research

Returns per-ingredient listings, source evidence, ranking and caveats. Draft creation invokes the research service internally. The active page has no standalone search button.

PATCH /api/drafts/[id]/lines/[lineId]

Supports editing an interpretation, skipping a line or accepting a pack mismatch using a draft version. The current cart page does not expose these actions.

POST /api/drafts/[id]/approve

Rechecks draft version and eligibility. Returns one stored simulated receipt for the approved draft.

What persists

The built-in node:sqlite DatabaseSync driver writes data/orders.sqlite by default. A drafts table stores a UUID, a JSON draft body and a nullable JSON receipt. A line records its source, product, requested amount, pack size, integer price in cents, case count and status. The image stays out of the database. The browser renders the receipt with localized USD formatting.

What happens if two requests race?

SQLite starts an immediate transaction for changes. The server checks the caller's version and rolls back on failure. A stale edit gets HTTP 409 and the current draft. Approval requires at least one ready line and zero unresolved lines. Repeating approval for the same version returns the stored receipt. Edits after approval are rejected. Total = sum of ready-line pack counts multiplied by their prices in cents.

Rehearse the demo

Use the printed two-line note for a predictable walkthrough.

1 Open the scan screen

Run the app with a SQLite-capable Node.js 24+ installation. Use npm ci for setup and npm run dev to start. Choose English review, enter a US ZIP such as 10011, and select Simplified Chinese as the note language.

2 Upload the sample

Use tests/fixtures/chinese-order.png. Point to the two lines and explain that the merchant can read the interface in English while preserving what the cook wrote in Chinese.

3 Check the transcription

If flagged rows appear, compare them with the photo and confirm the actual wording. If both readings agree, proceed to the editable transcription. The two lines should request one case each of eggs and oat milk.

4 Add to cart

Show the original note beside the matched eggs and oat milk. Explain that one case describes a different quantity for each product. These two whole-case matches come straight from the demo catalog. They do not demonstrate live supplier research.

5 Approve the simulation

Show the $78 total, click Approve simulated order, then point to the receipt reference, SKUs and prices. The receipt is the end of the product flow. Start a new list returns to intake.

Two corrections to the existing pitch notes

Perform corrections before Add to cart. The active cart screen shows an automatic selection or “No match for this ZIP.” It has no edit, skip, alternative-product or supplier-search controls. Its restart action clears the current list.

Both the local catalog and comparable storefront offers can price a draft. The sample uses local prices. A real storefront match may supply the chosen product and price for an unresolved line. The ZIP expresses the intended destination. Final delivery requires supplier confirmation.

Questions judges may ask

Short answers you can adapt in your own words.

Where is the AI?

Tesseract provides the first visual reading. A hosted vision model checks the image and draft through OpenRouter. Application code handles alignment, ingredient matching, unit conversion, pricing and order approval. A model never approves a purchase.

Why keep the handwritten note visible?

A normalized product name loses some of the cook's wording. Seeing the source beside the interpretation helps the buyer catch a unit, modifier or numeral that changes what they would buy.

What works if an external service fails?

With one working OCR reader, the buyer can review its output. With neither, the app offers manual transcription from the photo. Supplier timeouts can produce partial results. A usable catalog pack can still price a line. An unmatched line is left out.

How did you verify it?

The 52 server tests and the TypeScript check passed for this walkthrough. Tests cover OCR alignment, a real local scan of the printed sample, ingredient parsing, pack math, simulated supplier responses, version conflicts and receipt persistence. Browser scripts use Playwright and fixtures. Some target earlier review controls. This verification did not exercise the live hosted model or current storefront inventory.

How accurate is it?

The repo has example-based tests. It does not establish a handwriting accuracy rate across merchants or languages. The fixture is printed. A pilot could measure correction rate, time per list and surplus per item using real closing-shift notes.

What is the DoorDash connection?

The pitch is framed for the DoorDash merchant-experience track. The working demo ends at a locally stored simulated receipt. Real supplier accounts, purchase submission, payments, delivery and reordering would require further integration.

> ### A 20-second closing
>
> "We preserve the cook's words, show the owner where the readings disagree, and turn the verified list into whole packs they can review. The demo saves a simulated receipt. Our next test is whether this saves time on a real closing shift."